

Artifact Package Summary for ”Exploring and Improving Real-World Vulnerability Data Generation via Prompting Large Language Models”

This artifact package provides the executable scripts, pre-generated vulnerability samples, and analysis code used in the paper “*Exploring and Improving Real-World Vulnerability Data Generation via Prompting Large Language Models.*” It covers the full workflow from prompt-based LLM vulnerability sample generation, preprocessing, and statistical analysis, to downstream vulnerability detection/localization evaluation and baseline comparisons. For experiments that require re-generating samples, the artifact supports inference through a local model service (compatible with OpenAI/Ollama-style HTTP APIs). Since setting up LLM runtimes and external tools (e.g., CodeQL, Joern/Neo4j, and `clang-format`) can be non-trivial, reviewers can alternatively rely on the provided pre-generated archives and intermediate artifacts to reproduce the main statistics and comparisons without requiring GPU access or proprietary model credentials. Because some archives and third-party tool bundles are large, we recommend allocating sufficient disk space; and for deep-learning components (e.g., LineVul and FVD-DPM), GPU resources are recommended when training or large-scale inference is needed.

The experiments in this artifact can be organized into five parts corresponding to RQ1–RQ5 in the paper. **RQ1** provides the main injection/generation driver `RQ1/generate_sample.py` along with preprocessing, case-study, and statistics scripts to reproduce sample generation and quality analysis under different prompting modes (standard / few-shot / CoT). **RQ2** offers utilities for classifying edit types and separating “small/single-line” vs. “multi-line” changes (`divide_edit.py`, `divide_sm.py`, where the latter depends on `clang-format`) to

construct and analyze datasets by edit granularity. **RQ3** contains archived tool outputs (e.g., `VGX.zip`, `VulGen.zip`) so reviewers can inspect generated samples, metadata, and distributions without re-running the generators. **RQ4** provides CVE-oriented generated data (`CVE_GPT.zip`) and trigger-location mappings (`vul_trig_func_loc.csv`), together with integrated downstream tool bundles and baselines (LineVul, FVD-DPM, and CodeQL) for evaluating whether generated vulnerabilities benefit detection and localization. **RQ5** includes a BM25+LLM retrieval-augmented (RAG) baseline (`RQ5/test_baseline_rag.py`) that reports Top-1/Top-3/Top-10 localization accuracy and supports both zero-shot and retrieval-augmented settings.

In terms of required resources and runtime, lightweight reproduction (e.g., unzipping pre-generated samples, running statistics scripts, running RQ2 classification, and running the BM25 retrieval component in RQ5) can typically be done on a CPU-only machine. In contrast, compute-intensive steps (re-running LLM generation or training/evaluating deep learning models such as LineVul/FVD-DPM) may require substantial time, GPU support, and additional system-level dependencies. To facilitate reproducibility and reuse, each `RQ*/readme.txt` documents the entry scripts, input/output formats, and key parameters, and we recommend starting with small-scale smoke tests before running full-scale experiments.

This artifact is prepared for ICSE Artifact Evaluation with the goal of supporting the “Available” and “Reusable” badges, and can be further containerized to reduce setup overhead if needed. Reviewers only need basic command-line and Python skills; for parts involving CodeQL, `clang-format`, Joern/Neo4j, or a local model server, please follow the corresponding subdirectory READMEs to install and configure the required tools.